OPENAGENTS / TECHNICAL RESEARCH

Tinfoil

Company, products, architecture and integration

A practical assessment for Coder Cloud, Coax and Psionic

RESEARCH SNAPSHOT / NOVEMBER 11, 2026

Recommended entry point

Add verified inference behind Coder's provider interface. Evaluate confidential agent execution, key management and durable storage as a second, explicit product decision.

01 Company and product landscape

02 Current catalog, pricing and operational constraints

03 Trust boundaries, architecture and implementation path

Primary-source research and live pricing review. Includes 55 source links.
Architectural recommendations are identified separately from vendor claims.

1. Assessment and company

Recommendation: integrate Tinfoil first as a verified inference provider, then evaluate confidential agent execution separately. For OpenAgents, the immediate fit is a provider adapter behind Coder's configured model-access door. For Coax, use a verified local bridge or a narrow Rust boundary. Keep your Google Cloud systems in place while testing whether confidential workers solve a specific customer requirement.

Tinfoil combines hardware-isolated computation with client verification. Its commercial offer spans private chat, hosted inference, custom containers and a newer confidential development-machine product. The most important engineering question is where plaintext begins and ends. Using a verifying Tinfoil SDK from a conventional application server protects against inference infrastructure seeing content under the stated assumptions; it does not make that application server unable to read content. [3] [16] [29]

Three adoption decisions

Adopt for evaluation: verified hosted inference. Pilot separately: customer-private agent workers and encrypted memory. Require a concrete contract and deployment design: on-prem integration, production reliance on V3 beta features, or dedicated multi-GPU capacity.

Company fact	Verified finding
Identity	Tinfoil, Inc.; San Francisco. YC lists founding in 2024 and Spring 2025 batch.
CEO and cofounder	Tanya Verma: former Cloudflare engineer/researcher; contributed to Workers AI.
Cofounder	Jules Drean: MIT PhD in secure hardware; NVIDIA and Microsoft Research experience.
Cofounder	Sacha Servan-Schreiber: MIT PhD in cryptography/privacy; Microsoft Research and MongoDB experience.
Named angels	Paul Graham, Nick Sullivan, Alana Renda and Michael Grinich.
Unverified financials	No primary-source total funding, valuation, ARR or revenue established in this review.

Founding and founder information comes from YC and the company site. The latter names more people than YC's older headcount, so a precise current employee count would be misleading. [1] [2]

The company lists customers and projects including DuckDuckGo, Maple, Screenpipe, Syncopate Lab, Cybera, Mozilla, Workshop Labs, UC Berkeley, Harvard and Pour Demain. University logos should be read as project evidence, not proof of institution-wide purchasing. DuckDuckGo independently confirms Tinfoil serves GPT-OSS 120B and Gemma 4 31B in Duck.ai. [4] [5]

Business position

My assessment is that Tinfoil's differentiation lies in integrating the entire verification path: hardware, immutable workloads, model artifacts, encrypted transport and usable developer interfaces. The founders' backgrounds fit that problem unusually well. Its advantage is operational and systems engineering, rather than ownership of a frontier foundation model.

For OpenAgents it is both a potential supplier and, through Bunkers, an adjacent product. That argues for a replaceable provider adapter and clear ownership of your agent workflow, evaluation system, customer relationships and billing. Do not make Tinfoil-specific assumptions part of the Coax language or Psionic tensor APIs.

2. Products and supporting services

Offering	What it provides	Relevance
Chat	Web/iOS assistant with private inference and encrypted cloud history.	Evaluate product experience; reference for private memory.
Inference	Hosted OpenAI-compatible models with verifying SDKs.	Fastest route to confidential model access.
Containers	Your Docker workloads inside attested CPU/GPU confidential VMs.	Custom agents, RAG, model serving and training.
Bunkers - Preview	Persistent confidential development machines plus Perimeter policy enforcement.	Potential Coder Cloud execution backend.
Proxy / SDKs / CLI	Verified clients and a local OpenAI-compatible bridge.	Integration with existing applications and tools.
Enterprise	Custom agreements, white labeling, SSO, SLA and on-prem integrations.	Regulated or customer-specific deployments.

The homepage presents three core products; current documentation also includes Bunkers in Preview. Enterprise features are listed in the live pricing panel. On-prem integrations are advertised, but that wording does not establish arbitrary BYOC, air-gapped operation or a turnkey self-hosted distribution. [3] [6] [16]

Tinfoil Chat

Chat supports uploads, model selection, projects, encrypted history search, imports and shareable snapshots. Projects supply common instructions and documents while conversations retain separate histories. Deep research is documented as a Premium feature, with runs lasting up to about 20 minutes and one run per hour. Voice input and multimodal inputs depend on plan/model. [7]

The interface also offers personalization, custom prompts and generated interactive widgets. Web search and its automatic PII blocking are enabled by default in the documented clients. Browser and iOS controls differ. No Android client was confirmed from the current product pages. [9] [3]

History protection is more substantial than a conventional database encryption setting. A device-generated AES key protects synced chats, projects, attachments and indexes. It is supplied over an attested encrypted connection to the harness enclave for in-memory processing. Therefore 'Tinfoil does not possess a usable persistent plaintext key' is the useful claim; 'the key never leaves your device' would be inaccurate. Web local caches lack an additional application encryption layer. [8]

Passkey Kit can wrap a user's content key using WebAuthn PRF-derived material. It has JavaScript and Swift support and interoperable records. It manages key protection and recovery; your application still supplies authentication, record encryption and storage. This is independently useful if OpenAgents builds private history or files. [37]

Inference, models and auxiliary APIs

The available surface includes text/vision generation, function calling, structured responses, embeddings, audio, decision models, safety models and privacy filtering. Model support varies. Chat Completions and Responses both appear in current guides, but familiar client method names do not guarantee every OpenAI feature is implemented. [10] [11] [12] [13] [14] [38]

Service	Implementation / boundary
Document conversion	PDF/Office/HTML/CSV/images to Markdown, with enclave document processing and separate enclave OCR/vision; /v1/convert/file. Base64 file_data supported; file_id and /v1/files upload flow unsupported.
Web search / page fetch	Integrated search or Streamable HTTP MCP at https://websearch.tinfoil.sh/mcp. Search and fetch are exposed as tools; queries/URLs go to an external provider.
Privacy filter	POST https://pii-filter.tinfoil.sh/redact through a verified client. Returns detected spans and redacted text; response includes sensitive original content.
Decision models	Typed probabilities, choices and rubric scores. Useful for routing/evaluations; current aliases can change underlying checkpoints.
Admin API	Key management, usage, billing and container lifecycle. Keep admin credentials separate from inference credentials.
Verification tools	CLI verification and embeddable Verification Center for showing evidence to users.

Sources for the auxiliary API boundaries: document processing [35], MCP [36], privacy filter [15], decision models [13], Admin API [39], and SDK/verification documentation [16] [23]. PII detection can miss sensitive text; a successful filter response is not a proof of anonymity.

Containers and Bunkers

Containers expose Tinfoil's confidential runtime for your applications. The March 2026 launch describes application backends, proprietary models and training pipelines, including Workshop Labs usage. This is evidence of custom training support, not a separate managed fine-tuning API. GPU access is enabled by arrangement. [17] [18]

Bunkers are mutable Ubuntu development machines with encrypted persistent workspaces, Docker, SSH, browser terminals and optional CPU-rendered desktops. They include several coding agents and can connect to either Tinfoil inference or outside providers. Sending an agent's prompts to its conventional model vendor remains an external disclosure even when the workspace itself is confidential. [19]

The separate Perimeter enclave holds actual service credentials, gives the worker placeholders, controls destinations/actions and requests approval where policy requires it. This separation is useful for a software factory: an agent may need repository access without receiving an unrestricted reusable token. [21]

3. Current catalog and economics

Snapshot as of November 11, 2026. USD token prices below are per million tokens; a dash means no separate cached rate is listed. Model context figures are catalog limits, not a guarantee of latency, quota or effective reasoning quality. Prices come from the live official table; model IDs/context from documentation. [6] [10]

Model ID	Context	Input	Cached	Output
gpt-oss-120b	131K	$0.15	-	$0.60
gemma4-31b	256K	$0.40	-	$1.00
deepseek-v4-1-flash*	1M	$0.60	$0.01	$1.45
glm-5-3	1M	$1.80	$0.45	$5.75
kimi-k3-1-pro*	256K	$4.00	$0.40	$20.00
llama3-3-70b	128K	$1.75	-	$2.75

* Kimi K3.1 Pro is marked experimental on pricing; DeepSeek V4.1 Flash is marked experimental in docs. Kimi K3 remains listed as deprecated. Treat these statuses conservatively where pages differ. GLM-5.3 reasoning is always enabled; reasoning_effort controls it and its reasoning consumes the output allowance. [6] [10]

Additional model	Input / output	Purpose
nomic-embed-text	$0.016 / $0	768-dimensional embeddings
gpt-oss-safeguard-120b	$0.15 / $0.60	131K-context policy evaluation
pii-filter	$0.80 / $0	Sensitive span detection
voxtral-small-24b	$0.20 / $0.60	Audio understanding / transcription
decision-small / medium / large	$0.02 / $0.06 / $0.40 input; $0 output	Experimental 4B / 9B / 27B; 32K context

Other listed request fees: Whisper Turbo and Qwen3 TTS $0.01; Voxtral TTS and Realtime $0.002; document upload and web search $0.05. These are labeled per request, not per audio minute. Confirm duration/page limits for your workload. [6] Catalog details: embeddings [12], safety [14], decisions [13], audio [11].

Two integration gaps matter: Decisions currently has documented Python/Go/JS support but no Rust/Swift support; verified realtime speech WebSockets currently require tinfoil-js in Node rather than browser JS. Use a supported trusted component for those features and include it in the privacy boundary. [13] [11]

Compute pricing

Resource	Published hourly rate
CPU	$0.06/core
RAM	$0.01/GB
Provisioned disk	$0.000137785/GB
H200 / B200 / B300	$5.90 / $8.12 / $9.23 per GPU

Chat is $20/month. GPUs require monthly commitments despite hourly quotes. Disk remains billable until deleted. Bunker retail pricing was not verified. [6]

Calculated at an illustrative 730 hours/month: 4 vCPUs plus 16 GB RAM costs $292; 100 GB disk adds $10.06. One H200 costs $4,307 before CPU, RAM and disk; an eight-H200 GPU component alone is $34,456. These are arithmetic illustrations, not vendor quotes or utilization-adjusted comparisons.

For an API illustration, 10 million uncached input tokens plus 2 million output tokens on GPT-OSS costs $2.70 before tools, taxes and other fees. That quantity is a workload example, not an estimate of what your agents will use. Long tool traces and reasoning-heavy models can change the total substantially.

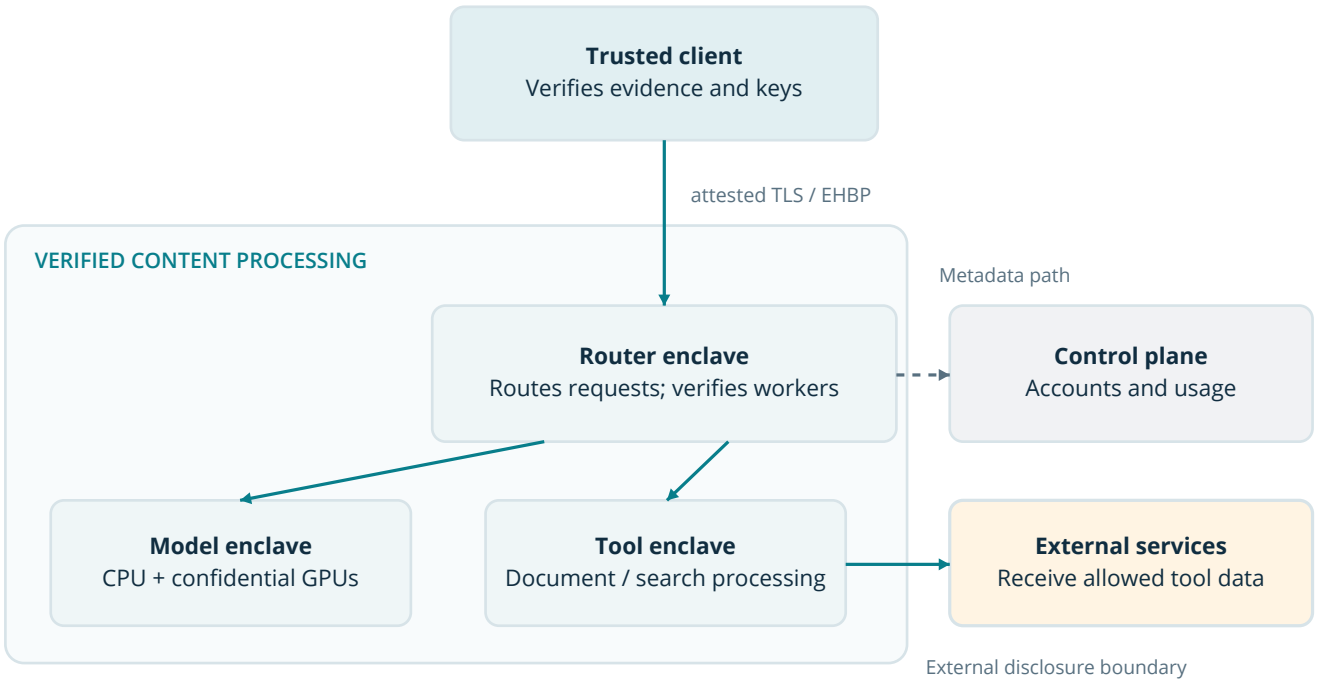
Budget and performance accounting

Spending caps are soft: admitted in-flight requests can overshoot them. Reasoning is counted as output and interrupted streams are charged for generated work. Record response usage against your own request ID; the Admin API's server-side records do not carry a client-supplied identifier for exact reconciliation. [39]

My recommendation is to meter cost per completed coding task, including failed attempts, retrieval, tool calls, retries and human review. Reserve budget before starting jobs, cap output and concurrency, then reconcile actual usage. Token price alone does not determine whether a provider improves your operating margin.

4. How the security architecture works

Tinfoil uses trusted execution environments (TEEs). Requests are encrypted in transit, decrypted inside a protected computation boundary, processed normally, and encrypted on their way back. This is different from fully homomorphic encryption, where computation operates on ciphertext. Tinfoil's design ultimately trusts supported hardware and the accepted workload software. [23] [48]



Documented router architecture, simplified. Solid connections carry content between verified components; the control plane receives operational metadata. Exact routing varies with client/proxy release. Sources: [24], [28].

Hardware, operating system and workload

The supported hardware combines AMD SEV-SNP or Intel TDX CPU isolation with NVIDIA Hopper/Blackwell confidential GPUs. Hardware attestation supplies evidence about the CPU/GPU configuration. Encryption of external memory and protected execution are intended to exclude the host and hypervisor from reading guest data during normal operation. [48]

The CVM is a small Linux environment assembled from pinned inputs with Nix. The root filesystem is read-only and dm-verity protected; supervisors, a container runtime, networking/shim services and GPU components make up the operational stack. The documentation describes a reduced kernel, restricted runtime privileges and an additive reproducible build. Rebuilding can independently test whether published source yields the released bytes. [25]

The documented inference router runs in an enclave and verifies downstream model/tool enclaves before forwarding plaintext over protected connections. Account management and billing remain outside this content boundary and receive usage information. Those orchestration components are closed source; 'security-critical components are open' is more precise than saying the whole commercial platform is open source. [24]

Verification chain

Check	Why it matters
Hardware evidence	A vendor-rooted report binds identity/configuration to a real protected environment.
Expected release	The verifier matches measurements against the allowed repository/build, not merely any signed enclave.
Build provenance	GitHub Actions and Sigstore establish publication and build provenance.
Independent reproduction	A rebuild can connect the public source to the published binary; a log alone cannot.
Connection-key binding	The actual TLS or HPKE key must belong to the verified workload.
Model-weight commitment	The model artifact loaded after boot must match its measured commitment.
Release policy / freshness	The application must decide which authenticated releases and evidence ages it accepts.

These are distinct checks. A legitimate signature does not establish that code is safe, and a known measurement does not prove an answer is correct. An application should fail closed on verification errors rather than retry through an unverified endpoint. V3 release pinning/freshness is currently beta and support differs by SDK. [23] [22]

Modelwrap and precise model identity

Modelwrap packages weights into a read-only artifact and commits a Merkle-tree root through measured configuration. dm-verity validates blocks as the inference server reads them. This closes a gap in ordinary boot measurement, which would otherwise say little about model files loaded later. It can verify the chosen quantized artifact, but it cannot prove benchmark quality, training-data provenance or semantic equivalence to another precision. [27]

For Psionic, the reusable design lesson is to make model identity explicit: source checkpoint, transformation/quantization, tokenizer, runtime build and expected tensor layout. A name such as a model-family alias is a weaker reproducibility target than an approved artifact commitment.

The October firmware change

Tinfoil's October 25 post describes replacing OVMF with roughly 300 lines of assembly and an IGVM build process. The kernel, initramfs and predetermined platform structures enter the initial hardware measurement directly. This reduces firmware parsing and runtime responsibilities; it does not reduce the entire trusted stack to 300 lines. General docs still reference OVMF, so the exact live release and migration coverage need confirmation. [26]

TLS, browser encryption and EHBP

Native clients can pin a TLS key to attestation. Browser clients need an additional mechanism because ordinary browser JavaScript cannot impose equivalent arbitrary TLS pinning. EHBP encrypts HTTP bodies using HPKE-derived keys while preserving routing headers. That lets an authenticated relay carry ciphertext; it does not conceal message sizes, timing or all metadata. Use the maintained implementation rather than inventing a new cryptographic protocol for Coax. [28]

The relay pattern also keeps the provider API key on your server. The client verifies the enclave, seals the request, and supplies an application-session token. Your relay validates that token, adds its provider credential and forwards the encrypted body unchanged. The receiver key remains bound to the enclave. [29]

5. Guarantees, limitations and maturity

Layer	Protection / remaining trust
Inference content	Designed to be inaccessible to Tinfoil/cloud operators when a verified client reaches the expected enclave.
OpenAgents application	If our ordinary backend receives plaintext, our infrastructure remains able to read it.
Client device	Browser code, extensions, native binaries, key stores and local storage remain trusted.
Tools and search	Allowed external services receive the data sent to them; an enclave does not make a remote API confidential.
Metadata	Account, payment, model/usage, traffic size and timing information can remain visible.
Application behavior	Attestation proves identity/configuration, not freedom from bugs, prompt injection or harmful actions.
Availability	The infrastructure can refuse service, stop machines or withhold storage.
Persistent freshness	Encrypted disks do not prevent replay of old sectors or restoration of an older disk; the application needs freshness and recovery controls.

The table combines documented privacy boundaries [49], the Bunker threat model [20] and the architectural consequences of choosing a trusted endpoint. It is the basis for precise product promises: name who is excluded from content access, not simply 'encrypted'.

Search, safeguards and other egress

The web-search enclave calls Exa, whose service reads queries or requested URLs under a zero-retention agreement. Tinfoil's privacy policy separately identifies context.dev for Chat page fetching. Thus search/fetch has a contractual external boundary, even where Tinfoil itself cannot inspect the content. Optional query redaction and prompt-injection filters reduce risk without proving complete filtering. [36] [38] [49]

Chat also runs enclave-based safeguard models. A violation flag and conversation identifier can leave the enclave and be associated with the account, while conversation content stays inside. This Chat mechanism should not be confused with a claim that every API/container workload is automatically moderated by the same pipeline. [54] [48]

Caching and tenant isolation

Prompt-prefix caches retain internal model state in volatile enclave GPU memory; they are not a persistent transcript database. They can survive a response until eviction but not an enclave restart. The user_cache_secret partitions cache reuse together with API identity. A shared backend should provide an opaque distinct value per user or allowed sharing group. It is neither an encryption key nor an erasure command; rotating it per call prevents hits without forcing immediate deletion. [34]

Current beta boundary

V3-dependent private secrets, encrypted volumes, attested keys/SSH, fresh container attestations and release pinning/freshness are beta. Some Go/Swift/Proxy/CLI verification paths share that status. Harbor/custom registry support is also beta. The feature-status page explicitly does not relabel inference and remote attestation as a whole beta; ordinary lifecycle, managed secrets and the S3 sidecar sit outside that blanket designation. [22]

The practical decision is release-specific. Document the exact SDK, attestation generation and feature combination a customer will run, then check that combination against your acceptance criteria. A stable inference endpoint does not imply every newer storage or key-management mechanism has the same support commitment.

Hardware limits and performance

TEE protection is conditional on hardware assumptions. TEE.fail demonstrates physical memory-bus attacks against relevant CPU trust foundations; it does not establish a Tinfoil compromise. Provider physical security and vendor vulnerabilities therefore still matter. A deployment should reject known-vulnerable configurations according to an explicit update policy. [53]

Tinfoil's own measurements report about 1% inter-token latency overhead in favorable CC-aware configurations, around 30% for naive compute-heavy inference, and over 100% in some low-concurrency cases. Training throughput losses were 18-32%. These are vendor benchmark results, not universal latency promises. The same analysis distinguishes protected GPU HBM from encrypted CPU DRAM and Hopper trusted-group NVLink from Blackwell encrypted links. [47]

Custom multi-H100/H200 deployment requires explicit acceptance of unencrypted NVLink. Select topology as part of the threat model rather than treating all confidential GPU configurations as interchangeable. [40]

Audits and compliance

Tinfoil reports completion of SOC 2 Type II and an infrastructure audit collaboration with Trail of Bits. This review did not obtain the complete audit reports, periods, findings or remediation evidence. Obtain those and check coverage of the exact release, especially newer V3, firmware and Bunker functionality. [48] [55]

The public terms target 99.9% monthly uptime but do not guarantee it. Contractual SLAs require a signed agreement; preview/beta commitments are excluded unless explicitly included. PHI and certain regulated processing require appropriate written agreements, including a BAA when applicable. This is a procurement condition in the published terms, not a result inferred from encryption alone. [50]

6. Integration into OpenAgents, Coax and Psionic

The recommendations below target your Rust Coder/Coder Cloud direction, Coax's C-compatible boundary and Google Cloud infrastructure. They do not require a PyTorch dependency in Psionic or a rewrite of your application around a Tinfoil SDK.

Three architectures with different promises

Architecture	Where plaintext exists	Appropriate promise
Server-side verified inference	User client, ordinary OpenAgents backend, verified Tinfoil workloads	Tinfoil/infrastructure cannot inspect model content under the stated assumptions.
Client encryption + opaque relay	User client and verified Tinfoil workloads; relay sees metadata	Our relay cannot read inference bodies; plaintext tools must execute elsewhere.
Confidential agent runtime	User client, verified OpenAgents worker, model/tool enclaves and explicitly allowed external services	The declared agent execution path protects content from infrastructure operators.

These are alternative product architectures, not merely increasing SDK settings. A ciphertext relay cannot privately inspect, rewrite, embed or execute the contents it cannot decrypt. If the backend must orchestrate tool calls on sensitive context, move that work into a verified worker or keep the loop on the user's device. The relay design and Containers make these choices possible, but the end-to-end application boundary remains ours to design. [29] [17]

A. Add a Rust provider adapter

Changing an ordinary OpenAI client's base URL reaches Tinfoil, but does not perform its connection-time attestation. Use a verifying SDK or the verified local proxy for that guarantee. Ordinary HTTPS retains the normal certificate checks, but does not verify the enclave's workload identity. [33]

Use the official Rust SDK behind the existing model-provider abstraction. It wraps async-openai and adds verified transport. The source is Apache-2.0 licensed. For Tinfoil-specific fields/events, the documented chat_relaxed interface avoids assuming the upstream OpenAI schema contains every extension. A valid SDK client does not automatically establish that every requested feature exists on every model. [30] [31]

```
use serde_json::json;
use tinfoil::Client;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Reads TINFOIL_API_KEY; verifies the enclave.
    let client = Client::new_default().await?;

    // A stable, random scope for this tenant/session.
    let cache_scope = std::env::var("APP_USER_CACHE_SCOPE")?;

    let request = client.chat_relaxed().request()
        .model("gpt-oss-120b")
        .messages([json!({
            "role": "user",
            "content": "Explain ownership in Coax."
        })])
        .set("user_cache_secret", json!(cache_scope));

    let response = client.chat_relaxed()
        .create(request).await?;

    if let Some(text) = response.content() {
        println!("{text}");
    }
    Ok(())
}
```

This is an illustrative integration sketch based on current documented interfaces, not an executed test. Pin a reviewed SDK revision and its dependencies. In a multi-user application, select cache scope per request from authenticated tenant state; do not let a process-wide default silently collapse all tenants into one cache namespace. [30] [34]

The adapter should normalize streaming text, reasoning, tool calls, usage and cancellation; expose model capabilities; enforce a privacy class on routing; and record verification outcomes. A confidential request should never fall back automatically to a normal provider after an attestation error. Keep changes in this adapter rather than spreading provider conditionals across the agent runtime.

B. Bridge Coax through an existing verifier

The first Coax integration should use the official proxy's loopback OpenAI-compatible endpoint, http://127.0.0.1:3301/v1. A separate process handles verification. The client-to-local-proxy segment is plaintext within the trusted machine; binding it to a public interface would change that boundary. The CLI is suited to headless applications; a desktop app is also documented. [32]

Later, if process boundaries or packaging matter, expose a narrow Rust C ABI: create client, start request, consume stream event, cancel and free resources. Keep allocation ownership, buffer lifetimes and callback threading explicit. IPC is easier to isolate and upgrade; an in-process ABI saves a process but brings the foreign runtime into the host failure boundary. These are engineering tradeoffs, not claims of an existing Tinfoil C SDK.

Do not begin by rewriting HPKE, certificate validation, vendor attestation, Sigstore verification and freshness policy in Coax. That would create a large security project unrelated to proving the product benefit.

C. Preserve billing while hiding prompts from our backend

```
import { TinfoilAI } from "tinfoil";

const client = new TinfoilAI({
  bearerToken: applicationSessionToken,
  baseURL: "https://our-relay.example/v1/",
  attestationBundleURL: "https://our-relay.example",
});
```

The documented relay can request X-Tinfoil-Usage-Metrics, delivered in a response header or streaming trailer. It carries model, token and cost information without prompt bodies. Preserve EHBP headers and streaming bytes, validate the destination, authenticate the user and replace the application token with the server-held provider key. Rust/Go proxy modes verify directly with the enclave, which affects network egress requirements. [29]

Our implementation should reserve budget before admitting work and reconcile the returned metrics with an internal request ID. Avoid putting prompts, documents, secrets or model responses in URLs, headers, analytics, traces or error reports. The point is a defined metadata contract, not a claim that the application collects no information.

D. Move sensitive orchestration into Containers

For a custom worker, build the Coax/Rust service as a Linux container, publish its immutable image, and create a public Tinfoil configuration repository. The image and source can remain private; the public config exposes the digest and measured deployment settings. Google Artifact Registry is supported. Do not assume a private registry makes the image invisible to every deployment component; negotiate model/source confidentiality requirements explicitly. [41]

The deployment sequence is: publish image digest; commit resources/routing; generate a measured release through the template workflows; deploy the chosen tag; verify the endpoint; then supply sensitive workload keys. Tagging or publishing a release alone is not the same operation as updating a live instance. [40] [45]

Operational constraint	Integration consequence
No inbound private networking	An external worker does not join our GCP VPC automatically; use authenticated service interfaces and review the network path.
No built-in load balancing	We own replica selection, health handling and attestable routing.
Ephemeral filesystem by default	Plan encrypted persistence and recovery before running durable jobs.
Public measured config	Keep secrets and confidential metadata out of the repository.
CPU / single-GPU stateless updates	Blue/green is available; test stream draining and job continuity.
Multi-GPU or persistent-volume updates	Replacement entails downtime; use checkpoints or application-level redundancy.
Debug deployments	They fail production attestation; use only non-sensitive development material.

These constraints come from the current Containers overview, deployment documentation and registry guidance, which are more specific than the marketing promise of automatic zero-downtime updates. [17] [45] [41]

E. Choose the right secrets and storage boundary

Ordinary managed secrets are encrypted at rest but accessible to Tinfoil infrastructure during deployment. If Tinfoil must be excluded from credential access, use the beta private-secret path: a keyserver you operate verifies the exact approved enclave/release before returning values over an attested channel. Keyserver availability becomes part of boot availability. [42] [43]

For shared durable objects, the buckets sidecar encrypts objects inside the enclave before S3 storage, using end-user keys. For a build tree, dataset or checkpoint filesystem, encrypted volumes offer one-enclave attachment and customer-held keys but are beta. Back up ciphertext and recovery material deliberately; state recovery is an application responsibility. [44] [46]

F. Evaluate Bunkers for Coder Cloud

Bunkers match mutable coding workloads better than a deliberately constrained serving container. A candidate integration would keep work orders, customer billing and orchestration in Coder, place the repository/tool execution in a Bunker, and use a Perimeter for credential injection and external operations. Evaluate its API/lifecycle and tenancy model before treating the preview as an embedded production backend. [19] [21]

The owner-key model is consequential: the documented Bunker has no ordinary account-based login and organization administration does not grant shell access. Its measured identity includes the owner public key and instance ID. That helps exclude infrastructure operators, but a shared enterprise coding service must deliberately design key recovery, collaboration and automation around it. [20]

A measured Nix toolchain is different from packages installed later. Mutated workspace software is not all covered by boot attestation. Malicious code inside a Bunker can read, change or delete workspace files; the Perimeter reduces credential/destination access, not every possible effect of hostile code. Outbound TLS interception is intentional so the Perimeter can identify actions and insert credentials. [20] [21]

G. Psionic and Google Cloud

Containers allow your own compiled service; they do not require Psionic to adopt Python or PyTorch. Prove CPU-container behavior first, then the supported CUDA/GPU path, checkpoint persistence, verification, and confidential-mode performance. Custom kernels and training harnesses need actual testing on the intended hardware; do not infer support from a Docker image building successfully.

For Psionic optimization, measure kernel-launch rate, synchronization, CPU/GPU readbacks, graph replay, batching and collective traffic. Confidential mode changes the cost of crossing boundaries, so a fast non-confidential execution plan may need different scheduling. Tinfoil's benchmark evidence is particularly relevant here. [47]

Google Cloud already offers confidential CPU/GPU infrastructure, but its current supported types and provisioning models differ by GPU; its docs list multi-node limitations for confidential GPU VMs. Building directly there means owning the application measurement, key release, client verifier, model-serving and public evidence workflow yourself. Tinfoil packages more of that workflow. Compare complete systems rather than a confidential-VM checkbox. [51] [52]

For your MMO and general services, keep simulation, realtime networking, authoritative game state and conventional infrastructure on the architecture chosen for those workloads. Use Tinfoil selectively for private development agents, sensitive support/operations tasks or customer-specific AI. The reviewed products do not establish a reason to move the whole game backend.

7. Adoption plan and commercial questions

Recommended sequence

Phase	Concrete work	Exit criterion
1. Provider evaluation	Add one Rust Coder adapter. Test selected models on our actual coding and document tasks.	Useful task success, latency and cost; verified failures remain failures; tool/stream/cancel behavior correct.
2. Privacy boundary	Choose ordinary backend processing or client encryption. Define allowed metadata and per-tenant cache scope.	A data-flow review identifies every plaintext component and every external recipient.
3. Confidential execution	Pilot one Container service or one Bunker workflow, including credentials and durable state.	Restart, update, key recovery, backup, cancellation and failure handling match the promised service.
4. Customer offering	Contract for required SLA, geography, capacity, support, resale and beta coverage.	Claims in the product and customer agreement match the actual verified path.

For the first model comparison, GPT-OSS provides a low-price reasoning baseline, Gemma supplies a vision option, and GLM offers a long-context coding candidate. Evaluate experimental models separately. These are sensible test categories derived from the catalog, not a claim that one model wins your workload. [10]

Use a fixed task set with representative repository sizes and tool chains. Record successful completion, regressions, time to first useful result, full task duration, generated reasoning/output, cache hit rate and total bill. Test malformed tool arguments, long outputs, cancellation and model-version changes because they create integration risk that a simple chat demo misses.

Record verification evidence in a way that does not leak content: accepted release identifier, model artifact/config identity, SDK build, verification status and usage receipt. Keep user-visible provenance separate from untrusted model narration. A model saying 'verified' is not evidence of verification.

Questions to settle before a production commitment

- Which SDK release, attestation generation, firmware path and model/router artifacts will our deployment actually use? What control do we have over upgrades and emergency revocation?
- Can the precise combination of private secrets, encrypted volumes and attested connections be supported contractually while V3 is beta?
- What regions and infrastructure providers will process our content and metadata? What regional controls cover inference, tools, storage, billing and support?
- What does 'on-prem integrations' include: hosted confidential inference connected to our data, our cloud account, customer premises, disconnected operation, or a narrower arrangement?
- What are Bunker pricing, access eligibility, automation interfaces, owner-key recovery and enterprise collaboration capabilities?
- What GPU capacity, topology and monthly commitments are available? Is our chosen multi-GPU interconnect encrypted, and what physical-security assumptions remain?
- Can we review the SOC 2 report and Trail of Bits scope, findings and remediation, including coverage of newer products?
- What are p50/p95 latency, concurrency, context, rate limits and service-credit terms for our selected models?

- May OpenAgents embed, meter and bill the services to its own customers under the proposed commercial agreement?
- What recovery, disk rollback/freshness, customer exit, encrypted-data export and key-management obligations remain ours?

What I would buy and what I would retain

I would buy a bounded amount of verified inference first and keep the integration replaceable. I would retain Coder's provider abstraction, customer policy, task evaluation, budget enforcement and evidence model. I would then pilot a confidential worker with a single well-defined workload before purchasing dedicated GPUs or depending on Bunkers for every coding job.

For a customer who mainly objects to an AI provider reading proprietary code, verified inference may solve the central objection with little platform change. For a customer who also excludes OpenAgents and cloud administrators from the trusted content path, the larger design must include client encryption, the agent loop, credentials, memory, storage and tools. That second offering is more work but creates a stronger and more defensible product promise.

The strongest conclusion is that Tinfoil is a credible candidate for the confidential-computation layer of OpenAgents. Its public engineering detail is unusually useful for evaluating the claim. The immediate adoption decision should still be based on our task quality, exact supported release and customer boundary, with the newer execution/storage products assessed on their own merits.

Primary sources

Sources accessed November 11, 2026. Links are clickable. Pricing reflects the live rendered official page; dynamic panels may not appear in plain-text extraction. Public documents can change after this snapshot.

This report uses primary company documentation, public source repositories, customer confirmation and hardware research. Prices were checked in the rendered official pricing page. Recommendations are architectural analysis; no paid inference, deployment, independent rebuild or performance benchmark was run.

[1] Company profile and founding details
https://www.ycombinator.com/companies/tinfoil

[2] Founders, team and named investors
https://tinfoil.sh/company

[3] Product overview
https://tinfoil.sh/

[4] Customer stories
https://tinfoil.sh/customers

[5] Duck.ai privacy and Tinfoil confirmation
https://duckduckgo.com/duckduckgo-help-pages/duckai/ai-chat-privacy

[6] Live pricing and billing FAQ
https://tinfoil.sh/pricing

[7] Chat features
https://docs.tinfoil.sh/chat/features

[8] Chat cloud sync and encryption
https://docs.tinfoil.sh/chat/cloud-sync

[9] Chat settings
https://docs.tinfoil.sh/chat/settings

[10] Chat model catalog
https://docs.tinfoil.sh/models/chat

[11] Audio model catalog
https://docs.tinfoil.sh/models/audio

[12] Embedding model catalog
https://docs.tinfoil.sh/models/embedding

[13] Decision model catalog
https://docs.tinfoil.sh/models/decisions

[14] Safety model catalog
https://docs.tinfoil.sh/models/safety

[15] Privacy filter API
https://docs.tinfoil.sh/guides/privacy-filter

[16] Documentation and product introduction
https://docs.tinfoil.sh/introduction

[17] Containers overview and limitations
https://docs.tinfoil.sh/containers/overview

[18] Containers launch, March 10, 2026
https://tinfoil.sh/blog/2026-03-10-tinfoil-containers

[19] Bunker features
https://docs.tinfoil.sh/bunkers/features

[20] Bunker security and threat model
https://docs.tinfoil.sh/bunkers/security

[21] Perimeters and access control
https://docs.tinfoil.sh/bunkers/perimeters

[22] Feature maturity and V3 beta scope
https://docs.tinfoil.sh/resources/feature-status

[23] How verification works
https://docs.tinfoil.sh/verification/verification-in-tinfoil

[24] Backend and attestation architecture
https://docs.tinfoil.sh/verification/attestation-architecture

[25] CVM trusted computing base and reproducibility
https://docs.tinfoil.sh/verification/cvm-image

[26] IGVM and minimal firmware, October 25, 2026
https://tinfoil.sh/blog/2026-10-25-less-is-more

[27] Model identity and Modelwrap
https://tinfoil.sh/blog/2026-02-03-proving-model-identity

[28] Encrypted HTTP Body Protocol
https://docs.tinfoil.sh/resources/ehbp

[29] Encrypted relay and usage metering
https://docs.tinfoil.sh/guides/proxy-server

[30] Rust SDK documentation
https://docs.tinfoil.sh/sdk/rust-sdk

[31] Rust SDK source and Apache-2.0 license
https://github.com/tinfoilsh/tinfoil-rs

[32] Local verified proxy CLI
https://docs.tinfoil.sh/local-proxy/cli

[33] Direct API access limitations
https://docs.tinfoil.sh/sdk/direct-api-access

[34] Prompt caching and tenant separation
https://docs.tinfoil.sh/sdk/prompt-caching

[35] Document processing and file limitations
https://docs.tinfoil.sh/guides/document-processing

[36] Confidential web-search MCP
https://docs.tinfoil.sh/guides/mcp-websearch

[37] Passkey Kit
https://docs.tinfoil.sh/guides/passkey-kit

[38] Integrated web search
https://docs.tinfoil.sh/guides/web-search

[39] Admin API, billing and reconciliation
https://docs.tinfoil.sh/admin/admin-api

[40] Container configuration and GPU topology
https://docs.tinfoil.sh/containers/configuration

[41] Private images and Google Artifact Registry
https://docs.tinfoil.sh/containers/private-images

[42] Managed secrets and environment variables
https://docs.tinfoil.sh/containers/secrets-and-env-vars

[43] Customer-controlled private secrets
https://docs.tinfoil.sh/containers/private-secrets

[44] Encrypted S3 buckets sidecar
https://docs.tinfoil.sh/containers/persistent-storage

[45] Container rollout and downtime rules
https://docs.tinfoil.sh/containers/updates

[46] Encrypted persistent volumes
https://docs.tinfoil.sh/containers/encrypted-volumes

[47] Confidential GPU performance study
https://tinfoil.sh/blog/2026-06-23-confidential-computing-overhead

[48] Security, privacy and compliance FAQ
https://tinfoil.sh/security-and-privacy-faq

[49] Privacy policy, October 12, 2026
https://tinfoil.sh/privacy

[50] Terms, availability and regulated use
https://tinfoil.sh/terms

[51] Google Cloud confidential VM supported configurations
https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/supported-configurations

[52] Google Cloud confidential GPU creation guide
https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/create-a-confidential-vm-instance-with-gpu

[53] TEE.fail hardware attack research
https://tee.fail/

[54] Chat safety and safeguards
https://tinfoil.sh/safety-and-safeguards

[55] Tinfoil statement on Trail of Bits collaboration
https://www.linkedin.com/posts/tinfoil_dan-guido-ceo-of-trail-of-bits-gave-us-activity-7435054465277325313-Vw6m